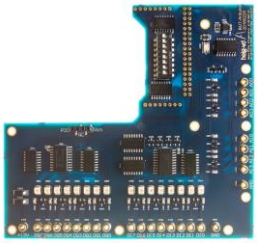
Laboratoire 4 - CAN

Introduction

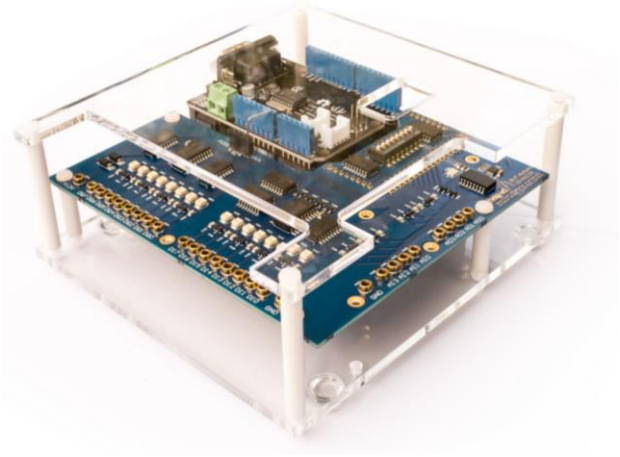
Application en laboratoire

Arduino Mega 2560

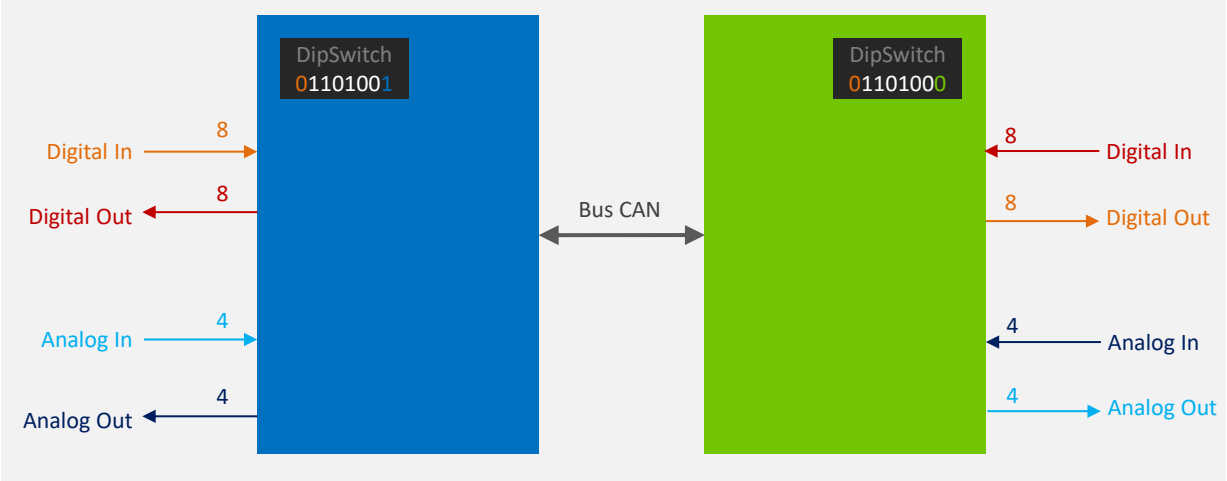
Shield CAN



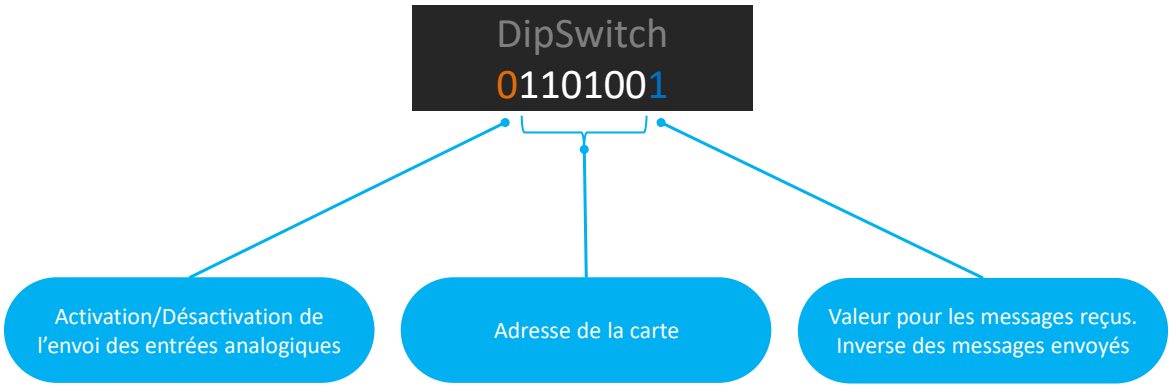
Carte IO HEIG-VD



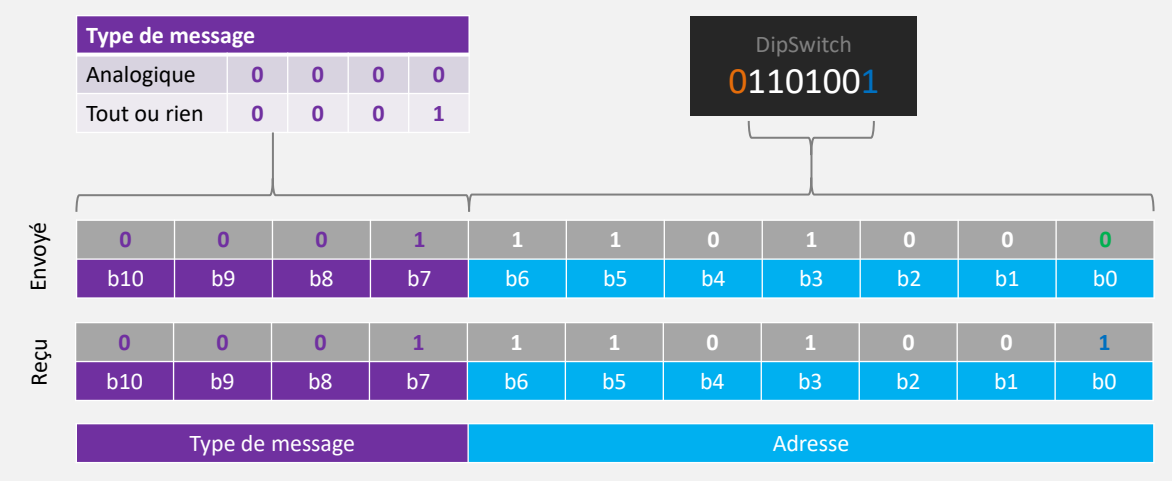
But du travail de laboratoire :
Ecrire le programme d'un module d'entrées sorties communiquant



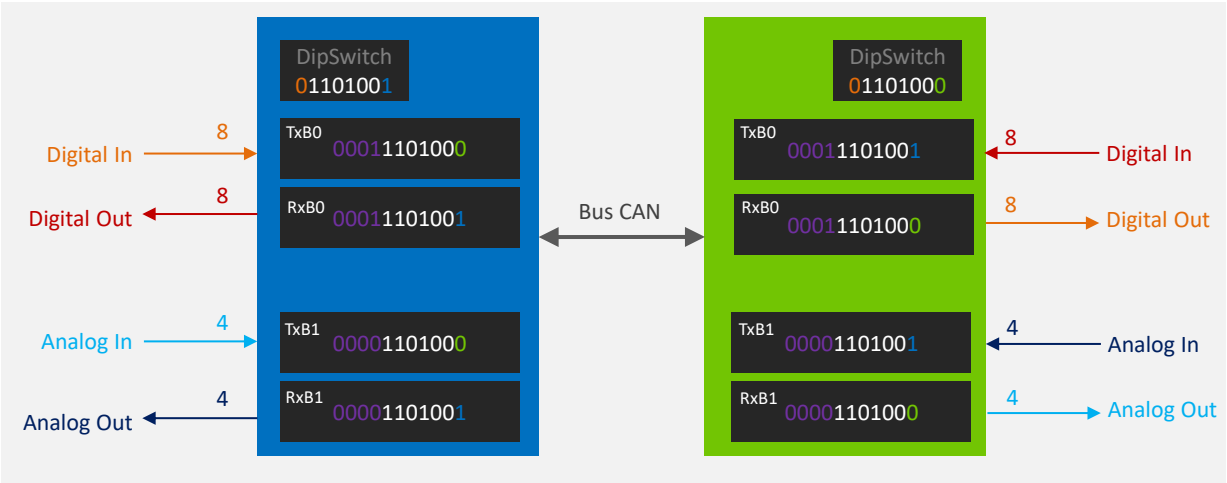
Manipulation de laboratoire
Fonctions à donner aux bits du DipSwitch



Manipulation de laboratoire :
Structure du champ d'arbitration 11 bits d'un message



Paramétrage du contrôleur CAN
Champs d'arbitration et registres du contrôleur



Format des messges

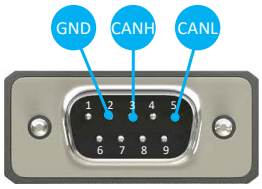
Les **7 bits** de 0 à 6 constituent des bits d'adresse.

Les **4 bits** de 7 à 10 constituent le numéro de message.

Les numéros de message suivants sont définis :

4 bits	Valeur hexa dans message	Données
0000	0x000 (+ **7 bits** d'adresse) En binaire : **000 0xxx xxxx**	Message contenant 4 valeurs analogiques (8 octets) : AIN0_LOW, AIN0_HIGH, AIN1_LOW, AIN1_HIGH, AIN2_LOW, AIN2_HIGH, AIN3_LOW, AIN3_HIGH
0001	0x080 (+ **7 bits** d'adresse) En binaire : **000 1xxx xxxx**	Message contenant 1 port 8 bits d'entrées/sorties tout ou rien (1 octet).

Confectionner le câble



- Relier GND, CANH et CANL
- Placer une résistance de terminaison/polaristion 120 ohms entre CANH et CANL

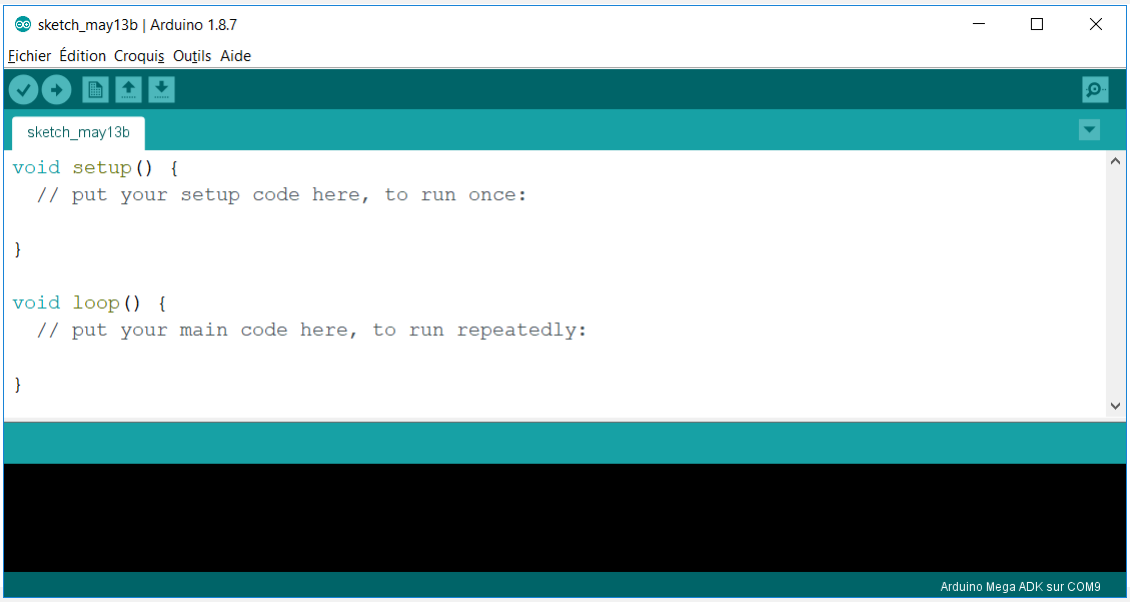
Prise en main Arduino

1. Récupérer les ressources sous Teams

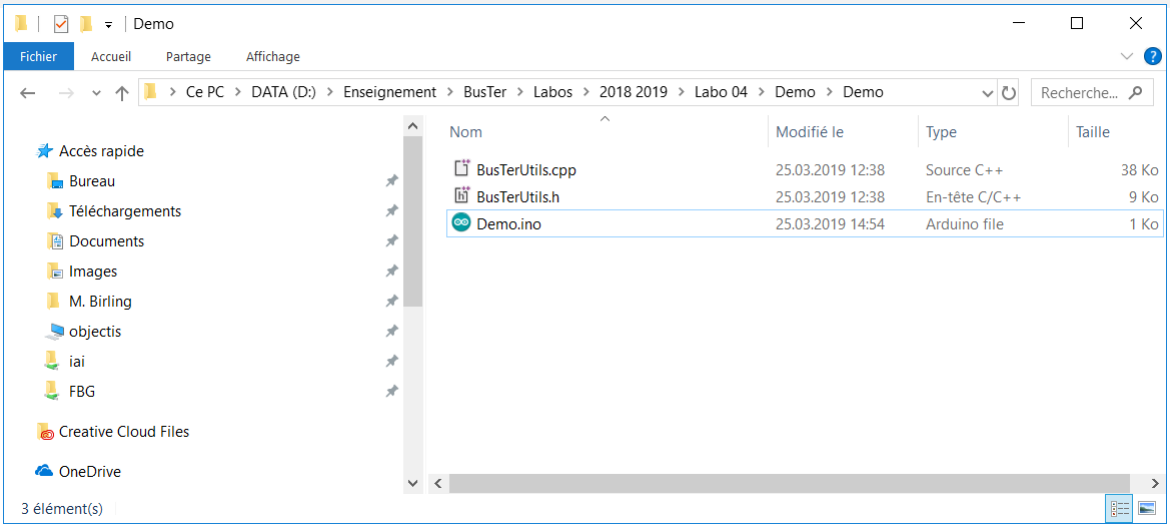
BusTer.Labo 04 - Ressources.zip

- ArduinoInterface.pdf
- BusTerUtils.cpp
- BusTerUtils.h
- MCP2515.pdf
- MCP2551.pdf
- MCP4728.pdf

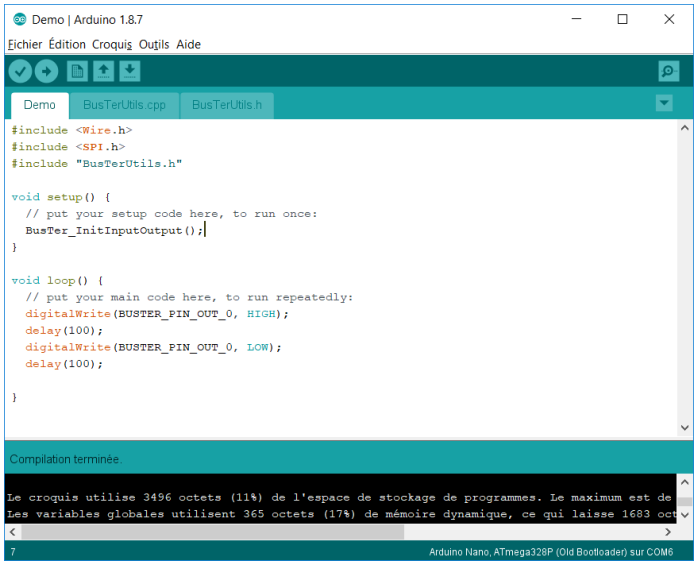
2. Installer et démarrer l'environnement Arduino



3. Enregistrer un projet arduino, placer les fichiers BusTerUtils dans le même dossier. Fermer et rouvrir le projet.

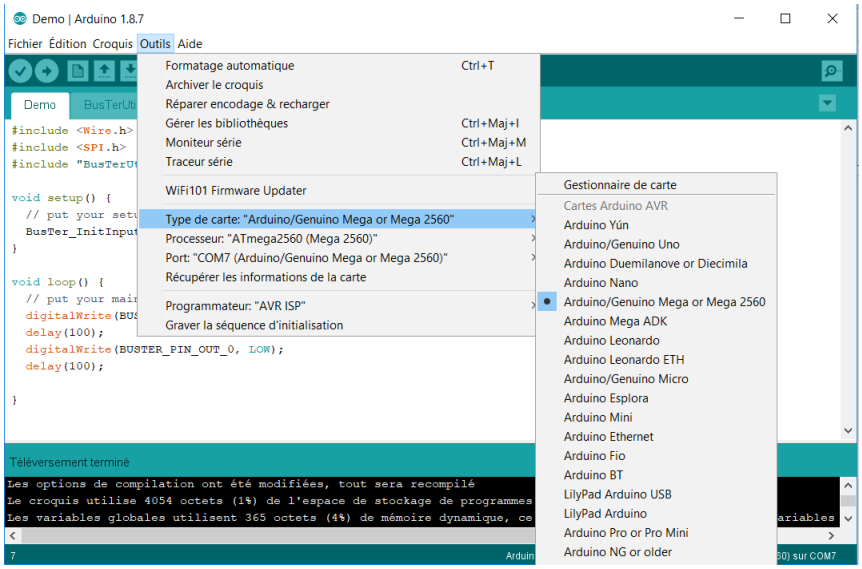


4. Les fichiers BusTerUtils s'ouvrent avec le projet. Saisir (ou copier/coller) le code source et compiler.

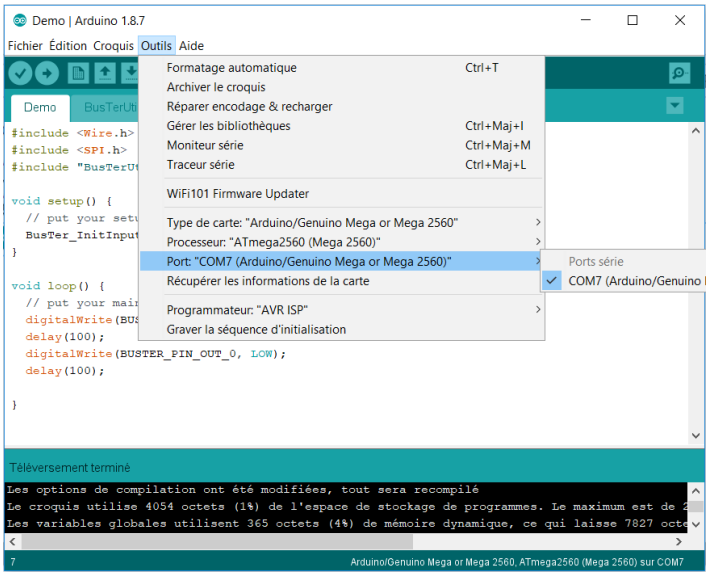



```
#include <Wire.h>
#include <SPI.h>
#include "BusTerUtils.h"

void setup() {
  // put your setup code here, to run once:
  BusTer_InitInputOutput();
}

void loop() {
    // put your main code here, to run repeatedly:
    digitalWrite(BUSTER_PIN_OUT_0, 1);
    delay(100);
    digitalWrite(BUSTER_PIN_OUT_0, 0);
    delay(100);
}
```

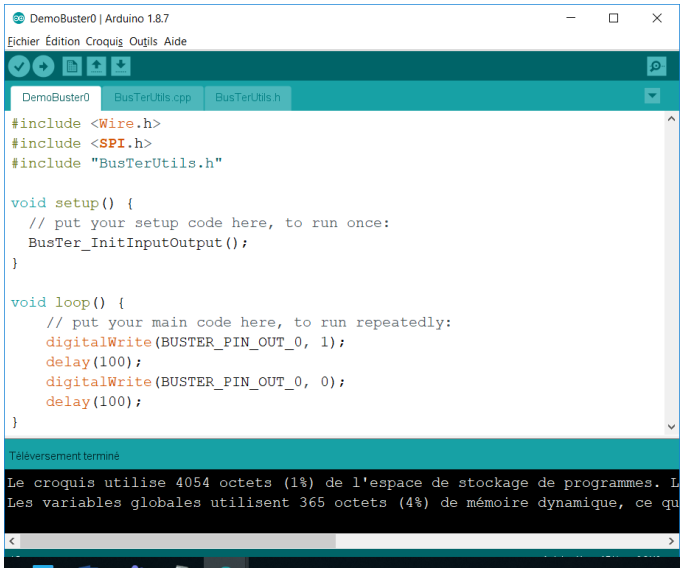

5. Sélectionner le type de carte : Arduino Mega



6. Sélectionner le port COM de la carte dans laquelle charger le programme



7. Compiler et «téléverser» le programme, vérifier que la led DQ0 clignote sur la carte BusTer



BusTerUtils contient les définitions et fonctions nécessaires pour le labo.

Demo - BusTerUtils.h | Arduino 1.8.7

Fichier Édition Croquis Outils Aide

Demo | BusTerUtils.cpp | BusTerUtils.h

```
// Define the pins used on Arduino

// Digital inputs
#define BUSTER_PIN_IN_0    40
#define BUSTER_PIN_IN_1    41
#define BUSTER_PIN_IN_2    38
#define BUSTER_PIN_IN_3    39
#define BUSTER_PIN_IN_4    36
#define BUSTER_PIN_IN_5    37
#define BUSTER_PIN_IN_6    34
#define BUSTER_PIN_IN_7    35

// Digital outputs
#define BUSTER_PIN_OUT_0 48
#define BUSTER_PIN_OUT_1 49
#define BUSTER_PIN_OUT_2 46
#define BUSTER_PIN_OUT_3 47
#define BUSTER_PIN_OUT_4 44
```

Compilation terminée.

```
Le croquis utilise 3496 octets (11%) de l'espace de stockage de programmes. Le maximum est de
Les variables globales utilisent 365 octets (17%) de mémoire dynamique, ce qui laisse 1683 oct
```

7 Arduino Nano, ATmega328P (Old Bootloader) sur COM6

BusTer_InitInputOutput doit être appelée avant tout accès aux entrées sorties de la carte pour leur configuration.

Demo - BusTerUtils.h | Arduino 1.8.7

Fichier Édition Croquis Outils Aide

Demo | BusTerUtils.cpp | BusTerUtils.h

```
// ***************************************************************************
// Function name: BusTer_InitInputOutput
// Descriptions:  Init the input and output of the arduino
// ***************************************************************************
void BusTer_InitInputOutput();

// ***************************************************************************
// Function name: BusTer_AnalogRead
// Descriptions:  Read an analog input corresponding to the input parameter.
// return value:  returns the input voltage in mV, range 0..5000.
// ***************************************************************************
int BusTer_AnalogRead(BUSTER_AnalogInput input);

// ***************************************************************************
// Function name: BusTer_AnalogWrite
// Descriptions:  Write the 4 analog outputs
//                Values are in [mV] in the range of 0..5000 mV
// ***************************************************************************
```

Compilation terminée.

```
Le croquis utilise 3496 octets (11%) de l'espace de stockage de programmes. Le maximum est de
Les variables globales utilisent 365 octets (17%) de mémoire dynamique, ce qui laisse 1683 oct
```

7 Arduino Nano, ATmega328P (Old Bootloader) sur COM6

BusTerUtils contient aussi les fonctions pour gérer le contrôleur CAN

Demo - BusTerUtils.h | Arduino 1.8.7

Fichier Édition Croquis Outils Aide

Demo | BusTerUtils.cpp | BusTerUtils.h

```
};

// Transmit buffers registers
enum BUSTER_CanTxb
{
    BUSTER_Txb0 = 0x30, BUSTER_Txb1 = 0x40, BUSTER_Txb2 = 0x50
};

// ****************************************************************************
// Function name:           BusTer_CanInit
// Descriptions:            Init the MCP2515 can controler with the canSpeed
//                          Set the controller in configuration mode
// ****************************************************************************
void  BusTer_CanInit(BUSTER_CanSpeed canSpeed);

// ****************************************************************************
// Function name:       BusTer_CanSetMask
// Parameter rxMask:    the mask register to set
// Parameter type:      Specification CAN_2.0A or CAN_2.0B
```

Compilation terminée.

```
Le croquis utilise 3496 octets (11%) de l'espace de stockage de programmes. Le maximum est de
Les variables globales utilisent 365 octets (17%) de mémoire dynamique, ce qui laisse 1683 oct
```

7 Arduino Nano, ATmega328P (Old Bootloader) sur COM6